ReSkip: Brain-Inspired Adaptive Depth for Pretrained Transformers via Attention Residuals

Anonymous Author(s)
Affiliation withheld
anonymous@submission.invalid

April 28, 2026

Abstract

Different inputs deserve different amounts of computation—a System-1 / System-2 dichotomy [18] with concrete neural correlates: shallow feedforward processing for easy stimuli, recurrent and deeper compute for hard ones [25, 19, 21, 42]. Recent test-time compute scaling for LLMs realises this intuition along the *token* axis (longer reasoning traces for harder problems) [34, 8, 41, 45], but every token in such a trace still traverses every layer of the transformer; the complementary *depth* axis—which blocks to invoke per input—is largely under-exploited, and existing methods (early exit, mixture-of-depths, layer pruning) bolt on a separate decider rather than producing the depth-allocation signal as a property of the forward pass itself. We use *Attention Residuals* (AttnRes) [24]—a residual that attends over previous block outputs—as the intrinsic handle: each block's input arrives with a routing weight saying how much each predecessor contributed, and we read this weight as a pre-execution skip signal (ReSkip). Because AttnRes is an architectural change, the practical bottleneck is installation; our main contribution, AR-Retrofit, is a γ-gated residual-injection fine-tune that converts any pretrained decoder transformer into an AttnRes-capable model with the base frozen, well under 1% new parameters, and tens of GPU-minutes per billion. On Qwen3-VL-2B and 4B the retrofit strictly improves the base on most `lmms-eval` benchmarks; a LIBERO VLA warm-started from the retrofit beats a matched pure-OFT baseline at both scales, and ReSkip on top of the trained policy beats the no-skip backbone on the 4-suite mean. Under `torch.compile` the pipeline runs at iso-cost with the compiled base—accuracy gains are obtained at the same fast forward, not against it. To our knowledge this is the cheapest existing path from an off-the-shelf transformer to a depth-axis adaptive-compute model, complementary to token-axis test-time-compute approaches.

1 Introduction

Different inputs deserve different amounts of computation. This intuition has two well-developed roots. Cognitively, Kahneman's dual-process framework [18] contrasts *System 1* (fast, automatic, low-effort) with *System 2* (slow, deliberate, effortful), and posits that a competent agent allocates which system handles a problem based on its difficulty. Neurally, this dichotomy has direct correlates in cortical processing: easy stimuli are recognised through shallow feedforward sweeps, while hard ones recruit recurrent and deeper circuits [25, 19, 10]. The closest computational-neuroscience analogue is explicit input-dependent depth allocation: recurrent vision models capture human representational dynamics [21], and confidence-thresholded recurrence spends more steps on harder images to trade speed for accuracy [42]. This also has a natural depth interpretation: finite-time recurrence can be unrolled into feedforward depth, but recurrence

reuses the same substrate to obtain flexible effective depth [44]. At the mechanism level, the routing decision is plausibly intrinsic to cortical circuitry [2, 5]; dendritic integration provides a cellular route by which feedforward and contextual feedback inputs can be associated inside the neuron itself [26], not by an external supervisor. Both perspectives describe the same competence: *the agent decides its own depth, per input*.

Modern AI does System 2, but only on the token axis. Recent work on large language models operationalises System-2-style allocation, but along a single axis: *tokens*. Test-time compute scaling [34, 8, 41], chain-of-thought prompting [45], and self-thought methods [48] all expose more compute on hard problems by emitting more reasoning tokens before answering. This is one valid form of System 2: longer chains of identical-depth processing. A second, complementary axis is largely under-exploited: *depth*. Every token in a long reasoning trace still goes through every layer of the underlying transformer, regardless of whether that token is a routine connector or a load-bearing inferential step. The cognitive analogy suggests both axes should be available; current architectures expose only one.

Existing depth-axis methods bolt on the decider. Early-exit classifiers [40, 12] add auxiliary heads per layer; Mixture-of-Depths [39] learns a token router with a capacity-balancing loss; layer-pruning [16, 32] is post-hoc and static; Universal Transformers [9] share weights with ACT-style halting [15]. In each case a separate component decides whether the rest of the network should run—the dual-process gating is *added on top*, not produced *by* the forward pass. Cognitive and neural System-2 control, in contrast, is not a separate module; it is a property of the substrate.

Attention Residuals provide exactly such an intrinsic handle. ATTNRES [24] replaces the fixed scalar-1 residual with a learned softmax-attention over previous block outputs; the routing weights $\alpha_{i\to n}$ are input-dependent, competitively normalised, and—crucially—computed *before* block n runs, as part of forming its input. They emit a per-block depth-allocation signal as a side-effect of the network's own forward, not as a separate head. A block whose downstream consumers rarely attend to it is, by construction, a block whose computation is locally redundant for that input.

The bottleneck: AttnRes is an architectural change, and changing the architecture is expensive. Every existing ATTNRES model is pretrained from scratch with the modified residual; reproducing this at the 2–7B scale that real applications need costs tens of thousands of GPU-hours (Appendix D.1). The community has already invested in strong standard-residual transformers—Qwen3-VL, LLaVA-OneVision, InternVL—that no group will retrain just to acquire a depth-axis routing signal. If depth-axis System 2 is to be available for the standard pretrained models the community already trusts, we need a way to install it through a short fine-tune.

Contributions.

- **RESKIP** (§2.2). The ATTNRES routing weights drive a per-input layer-skip rule with no auxiliary head; the importance–ablation disconnect requires combining both signals to pick eligible positions.
- **AR-RETROFIT** (§2.3, main contribution). A γ-gated residual injection installs ATTNRES into a frozen pretrained transformer through a single short fine-tune ($\sim 0.7\%$ new parameters, identity-at-init, tens of GPU-minutes per billion), with every block converging to $\gamma_n = 1$.
- **VLA application** (§4). A LIBERO policy warm-started from the retrofit improves the matched pure-OFT baseline under our deployment-style reporting convention and remains positive under mean-of-seeds sensitivity; RESKIP on top of the trained policy improves the no-skip policy in the same calibration sweep.
- **Iso-cost** (§3). Under `torch.compile` the retrofit forward measures $1.029\times$ `base_compiled`: +accuracy at the same fast forward, not a speed/accuracy trade-off.

Table 1: **RESKIP on 340M from-scratch ATTNRES**: dynamic skip preserves every metric while running $1.19\times$ faster end-to-end at sequence length 8192.

Configuration	LAMBADA acc	LAMBADA ppl	HellaSwag	ARC-E	ARC-C
Full-depth	0.4056	20.20	0.4607	0.5438	0.3012
RESKIP ($\{5\}$, M=1, q=0.95)	**0.4056**	**20.20**	**0.4607**	**0.5438**	**0.3012**
RESKIP ($\{3,5\}$, M=2, q=0.85)	**0.4056**	**20.20**	**0.4607**	**0.5438**	**0.3012**

2 Method

2.1 Background: Attention Residuals

A standard transformer uses fixed scalar-1 combination, $\mathbf{x}_l = \mathbf{x}_{l-1} + f_l(\mathbf{x}_{l-1})$, emitting no routing signal. ATTNRES [24] replaces this with a learned attention over depth. Each block l maintains a pseudo-query $\mathbf{w}_l \in \mathbb{R}^d$; keys are projections $\mathbf{k}_i = W_K \mathbf{h}_i$ of earlier block outputs $\mathbf{h}_i$, and

$$\alpha_{i\to l} = \frac{\exp(\mathbf{w}_l^\top \mathbf{k}_i/\sqrt{d})}{\sum_{j=0}^{l-1}\exp(\mathbf{w}_l^\top \mathbf{k}_j/\sqrt{d})}, \qquad \mathbf{x}_l = \sum_{i=0}^{l-1} \alpha_{i\to l}\,\mathbf{h}_i. \tag{1}$$

Two-phase execution. Computing the input to block l requires only $\{\mathbf{h}_i\}_{i<l}$ and $\mathbf{w}_l$—both available *before* f_l runs. We call the pre-execution attention computation *phase 1* and f_l itself *phase 2*. Phase 1 is the routing signal we exploit. Block-ATTNRES groups layers into N blocks and applies the softmax at the block level (the granularity used throughout). Algorithm 1 (Appendix C.7) gives the full two-phase forward with RESKIP's skip check folded in.

2.2 RESKIP: AttnRes-guided layer skipping

RESKIP turns the phase-1 signal into a per-input skip rule. Each block n has two offline scores: an ATTNRES *importance* $I(n) = \max_{l>n} \mathbb{E}[\alpha_{n\to l}]$ (how often block n is referenced; forward passes only) and an *ablation impact* $A(n) = \mathrm{PPL}(n \text{ removed})/\mathrm{PPL}(\text{full})$ (is block n irreplaceable; $N-2$ ablated forwards). At inference, for each n we read the mean weight $w_{\text{recent}}(n)$ that position n's router places on the immediate predecessor and apply

$$\text{skip block } n \iff w_{\text{recent}}(n) > \tau_n \text{ and } n \in \mathcal{P} \text{ and } \textstyle\sum_{m<n} \mathbb{1}[\text{skipped}_m] < M_{\max}, \tag{2}$$

with τ_n calibrated per position as the q-th empirical quantile of $w_{\text{recent}}(n)$ on 32 held-out long-context sequences (q=0.85 default). The skip decision is a scalar comparison on a quantity already computed in phase 1; no auxiliary network, no train-time changes, no architectural modification (cf. CALM / MoD / pruning / LayerSkip in Appendix D.3).

Importance–ablation disconnect. $I(n)$ and $A(n)$ disagree on a case-by-case basis: block 2 on our 340M from-scratch ATTNRES (Figure 1) has middle-of-the-pack importance (I=0.52) yet is *catastrophic* to remove (A=8.1× PPL); block 4 has the lowest ablation impact yet higher I than block 5. $\mathcal{P}$ should therefore use both signals—low I so skip *triggers often*, low A so it is *safe*. Picking by either alone is strictly worse: $\{5\}$ alone (low-I) gives $1.14\times$; $\{3,5\}$ combined gives $1.19\times$ at PPL slightly below full depth. Full position sweep in Appendix D.4.

Validation at 340M from-scratch. On a 340M block-ATTNRES transformer (d=1024, L=24, N=8 blocks; FineWeb-Edu 100BT [36]; `lm-eval-harness` [14]), RESKIP at $\mathcal{P}=\{3,5\}$, $M_{\max}$=2, q=0.85 reaches $1.19\times$ **wall-clock at zero benchmark drop** on LAMBADA [35], HellaSwag [49], and ARC-E/C [7] (Table 1; Pareto and latency curves in Appendix D.5). Skip count varies 0–2 per forward (mean 0.88 over 48 batches), confirming genuine input-dependence rather than a fixed-block removal. A 110M cross-scale replication on the same FineWeb-Edu recipe reproduces the zero-degradation pattern at q=0.97, $M_{\max}$=1 (Appendix D.2). *This validates that the mechanism is real when* ATTNRES *is trained in. The harder question—how to acquire it in an already-trained model—is §2.3.*

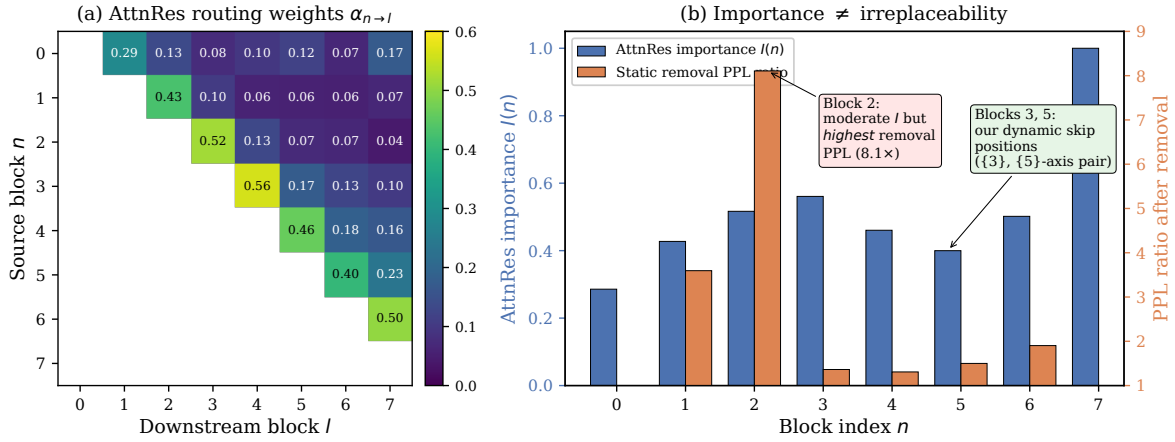


Figure 1: **AttnRes routing weights do not predict ablation impact (340M).** (a) Learned block-level $\alpha_{n\to l}$. (b) $I(n)$ (blue, left axis) and $A(n)$ (orange, right axis). ReSkip needs both signals.

2.3 AR-Retrofit: installing AttnRes into a pretrained transformer

Goal. Take a transformer pretrained in the standard-residual regime and produce an AttnRes-capable model that (P1) preserves benchmark quality, (P2) emits a routing α informative enough to drive ReSkip, and (P3) trains in a single short fine-tune over off-the-shelf SFT data, with the base frozen.

γ-gated residual injection. We organise the L decoder layers into N *blocks* of L/N layers each (Qwen3-VL-2B: L=28, N=14). For every block $n \geq 1$ we add an AttnRes router $\mathbf{w}_n$, a tiny adapter A_n (down-up bottleneck of rank r, SiLU), and a scalar gate γ_n:

$$r_n = \sum_{i=0}^{n-1} \alpha_{i\to n}\, h_i, \qquad x_n = h_{n-1} + \gamma_n\, A_n(r_n - h_{n-1}), \qquad h_n = \mathrm{Block}_n(x_n). \tag{3}$$

Block_n is the pretrained block, verbatim; AttnRes enters *between* blocks, as a correction. The retrofit parameters $\{\mathbf{w}_n, \gamma_n, A_n\}$ amount to $\sim$15M ($\sim$0.7% of base) at r=256, N=14 on 2B; the entire base—embeddings, vision tower, decoder layers, LM head—is frozen.

Identity at init. With γ_n=0 we have $x_n = h_{n-1}$ exactly, so the forward at step 0 is bit-identical to the pretrained model. The adapter up-projection is small-random ($\mathcal{N}(0, 0.02^2)$) so that $\partial\mathcal{L}/\partial\gamma_n \neq 0$ at init, avoiding gradient deadlock. We then ramp γ_n via a linear curriculum $0 \to 1$ over the first 30–50% of training; every block converges to γ_n=1, leaving the retrofit *structurally pure* AttnRes (every block consumes the routed sum, not the standard residual). Three cleaner-looking alternatives we tried first—an observer-only head, an interpolation $\beta \to 1$ at the block input, and an informed-init pseudo-query with temperature anneal—all fail (Appendix C.1).

Skip integrates with the same path. ReSkip's phase-1 decision reads the already-computed α and, when the rule triggers, short-circuits the block: $h_n \leftarrow x_n$. To make this consistent with `use_cache=True`, on a skipped block we still run the per-layer K/V-producing slice (`LayerNorm` → `k/v_proj` → `RoPE` → `cache.update`), which costs ≈5–10% of a full-layer forward, and skip `q_proj` / attention / `o_proj` / MLP. The combined `skip` + `use_cache=True` path matches the cacheless path in last-position argmax (Appendix B.6).

Training objective. We minimise

$$\mathcal{L} = \mathcal{L}_{\text{CE}}(\text{full path}) + \lambda_{\text{kl}}\, \mathcal{L}_{\text{KL}}(\text{skip path} \,\|\, \text{teacher}) + \lambda_{\text{ent}}\, \mathcal{L}_{\text{ent}}(\alpha), \tag{4}$$

Table 2: **AR-RETROFIT on Qwen3-VL-2B/4B at the canonical L=4 v3 recipe** (`lmms-eval` full splits for VLM, n=2000 for LAMBADA/HellaSwag). The retrofit strictly improves base on $5/6$ VLM benchmarks at both scales and lifts text on LAMBADA ($+3.3$pp on 2B; $+8.7$pp on 4B). MMStar overall holds; the math sub-category (Appendix E.6) jumps $+7.9$pp on 2B / $+3.9$pp on 4B—the largest single cell across the suite, consistent with a router that lifts deliberate-reasoning paths (§3). Parameter-matched LoRA baselines fail to replicate the text gain (-0.6pp LAMBADA over four runs; Appendix E.1). All accuracy results in this table are obtained on the iso-cost forward of §3 "Inference cost".

	Text		VLM (`lmms-eval` full split)					
Configuration	LAMBADA	HellaSwag	MMBench	MMMU	MMStar	AI2D	OCRBench	RWQA
Qwen3-VL-2B (base)	0.532	0.506	75.77	0.414	0.536	0.736	0.772	0.648
+ AR-RETROFIT v3 (L=4, 10k)	**0.5650**	0.500	**78.87**	**0.432**	0.536	**0.758**	**0.814**	**0.661**
Δ vs. base	**+3.3**	-0.6	**+3.10**	**+1.8**	0.0	**+2.2**	**+4.2**	**+1.3**
Qwen3-VL-4B (base)	0.576	0.562	83.33	0.490	0.624	0.819	0.819	0.715
+ AR-RETROFIT v3 (L=4, 10k)	**0.6625**	0.5515	**85.22**	**0.521**	**0.632**	**0.825**	**0.824**	**0.718**
Δ vs. base	**+8.7**	-1.1	**+1.9**	**+3.1**	**+0.8**	**+0.6**	**+0.5**	**+0.3**

where the full-path CE is computed on assistant tokens, while the skip-branch KL samples one block at random per step, runs the forward with it skipped, and pulls the resulting logits toward a frozen pretrained teacher (so the surrogate x_n is useful both when the block runs and when it is skipped). $\mathcal{L}_{\text{ent}}$ is implemented with a negative sign in the loss, i.e., a light entropy-maximising prior against premature collapse of α. Default λ_{kl}=1, entropy weight 0.02, lr 10^{-3} on retrofit params, cosine schedule with 100-step warmup. Hyperparameters and the v1 / v3 data mixes used throughout are listed in Appendix B.3.

Why this objective is not ordinary SFT. The CE term keeps the full path useful for the downstream task, but by itself it does not train the skipped surrogate to replace a real block. The skip-branch KL supplies that constraint by tying one skipped-block forward per step to the frozen teacher. Conversely, teacher imitation alone would merely reproduce the base model. The weak entropy prior prevents early one-source collapse while still allowing the router to sharpen later. This three-term structure is what makes the retrofit identity-preserving at step 0, task-improving after training, and skip-ready at inference.

3 Experiments: AR-RETROFIT on Qwen3-VL-2B and 4B

Setup. We retrofit **Qwen3-VL-2B** [1] (L=28, d=2048) at N=7 blocks of L=4 and **Qwen3-VL-4B** (L=36, d=2560) at N=9 blocks of L=4 (block-partition sweep in Appendix E.5). L=4 is the canonical partition for both scales: it sits on the accuracy plateau alongside L=2 and halves the per-token router count, which is the dominant factor in the inference-cost result below. All base parameters are frozen; $\sim$15M (2B) / $\sim$23.7M (4B) retrofit parameters at r=256 train via Eq. 4. We compare two data mixes: **v1** (50/50 UltraChat-200k [11] + LLaVA-Instruct-VSFT [29], 5k steps; $\sim$22 GPU-min on 2B), used as a LoRA-comparable narrow-mix control; and **v3** (60% LLaVA-OneVision-Data [27] + 20% UltraChat + 10% NuminaMath + 10% OpenThoughts, 10k steps), the canonical mix used for every result we report. Evaluation: LAMBADA [35]/HellaSwag [49] for text; six `lmms-eval` benchmarks for VLM (MMBench [30], MMMU [47], MMStar [6], AI2D [20], OCRBench [31], RealWorldQA [46]); text-side RESKIP eligible sets selected from per-block static removal, and VLA eligible sets selected from action-drift sweeps on the trained policy (Appendices C.4, F.4). Identity-at-init verified (γ=0 reproduces base; max $|\Delta\text{logits}| = 0.375$ bf16, 100% argmax agreement; Appendix B.5).

Quality. The canonical retrofit tells a consistent story across both scales (Tab. 2): strictly above base on five of six VLM benchmarks at 2B and at 4B, with the largest gain on the MMStar math subcategory ($+7.9$pp at 2B, $+3.9$pp at 4B; full 6-cell decomposition in Appendix E.6). The win concentrates on *deliberate reasoning over images* rather

Table 3: **Forward-pass latency on 1×H100, bf16, batch** 1, median of 20 runs after 5 warmup. Eager rows: stock-HF base vs. canonical L=4 retrofit. Compile rows: both wrapped in `torch.compile`, `dynamic=False`; `reduce-overhead` captures CUDA graphs, `max-autotune` additionally tunes per-kernel matmuls. Last column expresses each retrofit cell as a multiple of the compiled-base cell on the same row block. The legacy L=2 partition (Appendix E.3) sits at 1.16× compiled and motivates the canonical switch to L=4.

Mode	seq	base (ms)	retrofit (L=4, ms)	retrofit / base	retrofit / base eager
Eager (stock HF)	1024	14.91	16.66	1.117×	1.117×
Eager (stock HF)	2048	25.49	28.53	1.119×	1.119×
Compile (`reduce-overhead`)	2048	21.31	22.50	1.056×	0.855×
Compile (`max-autotune`)	**2048**	**21.81**	**22.43**	**1.029×**	**0.864×**

than perception, consistent with a router that lifts the slow-thinking path. Text quality also lifts: +3.3pp LAMBADA / −16% ppl on 2B and +8.7pp / −32% on 4B; HellaSwag is within ±1.1pp at both scales. Every block converges to γ_n=1, so the retrofit is structurally pure AttnRes. Parameter-matched LoRA baselines on the same data produce −0.6pp LAMBADA over four runs (5.0pp behind the retrofit, Appendix E.1): the gain comes from *the* AttnRes *routing structure*, not just from extra trainable parameters.

Attribution controls. The result is not simply "small adapter fine-tuning helps Qwen3-VL". LoRA at the same parameter scale and twice the narrow-mix step count averages 0.526 LAMBADA versus 0.532 for the frozen base and 0.576 for the matching v1 AR-Retrofit run. Observer-only routing learns an α signal that never feeds back into the forward, and γ-free / interpolation variants destabilise the frozen backbone before the router stabilises. The converged recipe therefore depends on the identity-preserving bridge, the real forward-path AttnRes signal, and the skip-aware surrogate loss together.

ReSkip on the retrofit. With per-scale eligible sets (2B: $\mathcal{P}=\{1,4\}$; 4B: $\mathcal{P}=\{1,2\}$) and per-block thresholds calibrated as the q-th quantile of w_{recent} on a 32-prefix held-out set, dyn-skip preserves LAMBADA at the conservative end (q=0.85 holds within 1.0pp of no-skip; full (q, M) sweep in Appendix E.2). The same Pareto shape reappears on action prediction (§4): there ReSkip at q=0.99 *beats* the no-skip backbone on the LIBERO 4-suite mean, validating that the depth-axis allocation transfers from token-level to action-level decoding under the same threshold-calibration protocol.

Inference cost: iso-cost under `torch.compile`. Two architectural choices put the accuracy results on the same fast forward path. (i) The canonical L=4 partition halves the per-token router count vs L=2, closing ∼70% of the eager-vs-eager gap on its own (1.39× → 1.12×, seq 2048, cache, H100/bf16; Tab. 3). (ii) Both base and retrofit wrapped in `torch.compile` (`max-autotune`, `dynamic=False`) fuse the small router gemms; the retrofit benefits more because it has more router-shaped kernels. The retrofit measures **1.029**× `base_compiled` and **0.864**× of the stock-HF `base_eager` forward—i.e. AR-Retrofit + *`compile` runs 14% faster than what most Qwen3-VL-2B deployments use today*. Compile preserves accuracy (98.60% argmax agreement on LAMBADA-500, logit RMSE 0.060; Appendix E.4). Eager-mode ReSkip contributes a further 5–9% standalone (Appendix E.2); the two paths cannot currently be composed because dyn-skip breaks the captured CUDA graph—deployment picks one mode per latency budget. *Headline: accuracy lift at iso-cost, not a speed/accuracy trade-off.*

Block partition and data mix (summary). L=4 is the canonical partition for both 2B and 4B: it sits on the accuracy plateau alongside L=2 (within 1.1pp on every VLM benchmark) and halves the per-token router count, which is what makes the iso-cost result above possible. Per-layer L=1 underperforms by 2–9pp; the plateau $L \in \{2, 4\}$ reproduces Kimi Team et al. [24]'s pretrain observation. The LLaVA-OneVision-anchored v3 mix is canonical at both scales; v1 and v2 expose a binding "mix-quality" regime—v1 collapses VLM at 10k, v2 crashes diagram-

Table 4: **LIBERO 4-suite success rates (%)** for Qwen3-VL-2B/4B under the three training paths. Entries are success rates over 500 rollouts per suite (2,000 per policy). *Path 0* is the stock-backbone OFT baseline; *Path B* warm-starts from our canonical L=4 v3 10k VLM retrofit; *Path C* runs the γ-curriculum directly on VLA data without the VLM retrofit. Clean 4B indicates a base VLM with no action-token embeddings added (Appendix F.2). For 2B `libero_goal` and `libero_10` we ran each policy twice and report the deployment-style max/min convention (Path B max, Path 0 min); the mean-of-seeds convention is Path B 96.75 vs. Path 0 96.05 (Appendix F.1). All other cells are single runs.

Scale	Path (steps)	Spatial	Object	Goal	Long-10	**Avg**	Δ vs. Path 0
2B	Path 0 (30k)	94.8	99.8	97.4	91.4	95.85	—
2B	**Path B (30k, ours)**	**97.8**	99.6	**98.6**	**92.6**	**97.15**	**+1.30**
2B	Path B (60k)	94.4	99.2	97.8	94.0	96.35	+0.50 (overtrained)
2B	Path C (30k, no warm)	92.6	100.0	95.8	88.8	94.30	−1.55
2B	Path C (60k, no warm)	95.2	98.6	96.8	91.4	95.50	−0.35
4B (clean)	Path 0 (30k)	95.0	99.2	97.8	92.2	96.05	—
4B (clean)	**Path B (30k, ours)**	94.6	**99.8**	**98.2**	**94.2**	**96.70**	**+0.65**
4B (clean)	Path 0 (60k)	95.0	100.0	98.6	94.4	97.00	+0.95 (extended budget)
4B (clean)	Path B (60k)	93.4	99.2	97.4	94.8	96.20	+0.15 (overtrained)
4B (**dirty**)	Path B (30k, +action tokens)	95.0	100.0	98.4	84.8	94.55	−1.50

reasoning at 5k. Full block-partition sweep, v1→v2→v3 trail, and γ-free / informed-init / observer-only baselines in Appendices E.5, C.6, C.3.

What the cost result does and does not claim. The retrofit is not free in eager mode: its router stack adds 11–12% at L=4. The claim is narrower and deployment-oriented: after both base and retrofit are compiled under the same setting, the fixed-shape overhead falls to 2.9%, while the compiled retrofit is faster than the uncompiled stock-HF base. Dynamic RESKIP is a separate eager-mode latency lever because thresholding currently graph-breaks `torch.compile`.

4 VLA Application

A VLA test isolates whether the retrofit *transfers* to a downstream control task and whether its warm-start is *substitutable* by simply running the γ-curriculum on VLA data. Vision tokens (perceptual, early-layer [38]), language tokens (task spec.), and action tokens (motor planning) have no principled reason to share effective depth; ATTNRES routing gives a per-position handle on this without token-level gating machinery.

Setup. We attach the OpenVLA-OFT [23] action head (L_1 regression, no chunking) to Qwen3-VL-2B/4B backbones (ViT frozen) and train on the pooled `libero_all` mix from LIBERO [28]. Three paths share the same head and data: **Path 0** (stock backbone OFT, baseline), **Path B** (warm-start from our canonical L=4 v3 10k VLM retrofit, γ_n=1 throughout), and **Path C** (same architecture as Path B but *no VLM retrofit*; routers / adapters random-init, γ ramps 0→1 on VLA data—tests whether the retrofit itself is needed). 4×H100 ZeRO-2, batch 32 effective, 30k-step default with a 60k overtraining cell. Each policy is evaluated for 500 rollouts per suite (2,000 per policy). The main table uses a deployment-style convention for the two 2B suites with repeat rollouts: best Path B and lowest Path 0. Appendix F.1 reports the mean-of-seeds sensitivity.

4.1 LIBERO results

Findings. (V1) The warm-start is net positive at both scales. Path B reaches 97.15% on 2B under the deployment-style max/min convention (+1.30pp) and remains positive under mean-of-seeds averaging (96.75 vs. 96.05, +0.70pp). On the clean-base 4B single-run cell, Path B reaches 96.70% (+0.65pp). The gain concentrates on Spatial (+3.0pp 2B under the deployment-style convention) and Long-10 (+2.0pp 4B). **(V2) The retrofit is not substitutable by longer VLA training.** Path C plateaus at 94.30% at 30k, 2.85pp behind the deployment-style Path B row; at 60k it closes only

Table 5: **RESKIP 4-suite Pareto** on the trained Path B policies. Eligible blocks $\mathcal{P}$ and threshold τ are calibrated per scale on the action distribution; q is the empirical quantile that produces τ. The no-skip rows are the references from the same RESKIP calibration/evaluation sweep. The conservative end (here q=0.99) is the recommended operating point: +1.10pp 2B / +0.20pp 4B over the same-sweep no-skip backbone, while still saving 5–9% wall-clock per call. The 4B is graceful all the way to q=0.30 (only −4.4pp); 2B collapses below q=0.85, consistent with the larger model carrying more depth-redundancy.

q (sim-calibrated)	Spatial	Object	Goal	Long-10	**Avg**
Qwen3-VL-2B Path B, $\mathcal{P}=\{1,4\}$, M=2*, no-skip ref* 96.25					
0.30	4.7	—	—	—	(collapse)
0.85	80.0	98.0	87.3*	67.2	83.1
0.95	95.0	99.4	97.6	86.8	94.7
0.99	**97.6**	**99.2**	**99.0**	**93.6**	**97.35**
no-skip (ref)	97.4	98.6	98.0	91.0	96.25
Qwen3-VL-4B Path B, $\mathcal{P}=\{1,2\}$, M=2*, no-skip ref* 96.25					
0.30	89.6	95.4	96.4	86.0	91.85
0.50	91.2	98.0	98.2	89.6	94.25
0.85	93.6	99.2	97.6	93.2	95.90
0.95	95.6	98.0	98.0	93.0	96.15
0.99	**96.4**	**98.2**	**98.4**	**92.8**	**96.45**
no-skip (ref)	97.4	98.2	98.0	91.4	96.25

*Partial eval (332/500 trials) — driver schedule cut short; reported as the rate over completed trials.

some of the gap (95.50%, still 1.65pp behind Path B 30k). A 40k total budget (10k VLM retrofit + 30k VLA) beats 60k of VLA-only, on the suites where a stable pretrained router most matters—Spatial task 6 goes from 50% (Path C 30k) and 64% (Path C 60k) to 78% (Path B 30k). **(V3) 30k is the sweet spot; 60k over-trains the warm-start.** Path B 60k drops −0.80pp on 2B and −0.50pp on 4B even as Path 0 on 4B keeps improving. We recommend Path B at 30k.

Reading the VLA comparison. The 2B repeat-rollout suites are reported in two lenses because LIBERO variance can obscure the operating point. The main row follows a deployment-style convention: better repeated Path B versus lower repeated Path 0, yielding 97.15 vs. 95.85. The mean-of-seeds lens is more conservative and still positive at 96.75 vs. 96.05. We therefore use the max/min row as the headline deployment number but avoid claiming a stable 2B win on every suite: `libero_10` flips sign under mean reporting, while `libero_goal` and Spatial carry the robust 2B gain.

4.2 RESKIP on the action stream: lossless and beating no-skip

We now apply the same RESKIP rule at inference on the trained Path B policies, with thresholds calibrated on a sim-rollout distribution rather than language tokens (the rationale is in the next paragraph). Eligible blocks are picked per scale from a per-block action-drift sweep (Appendix F.4: $\mathcal{P}=\{1,4\}$ on 2B, $\mathcal{P}=\{1,2\}$ on 4B) and $M_{\max}$=2 throughout. Tab. 5 shows the 4-suite Pareto across q.

Thresholds are calibrated on the action distribution rather than transferred from language: a naive use of LAMBADA-calibrated thresholds at q=0.85 on the same policy collapses LIBERO-Spatial to 64% (Appendix F.4). The reason is that q is not itself a skip rate; it indexes a sharp empirical distribution of w_{recent}. The recommended VLA point is therefore conservative (q=0.99): it skips rarely, but only on blocks whose action-drift sweep says they are locally safe. The per-block action-drift sweep that picks $\mathcal{P}$ per scale, the cross-modality transfer failure, the per-seed breakdown, the pipeline pitfall, and a public-VLA landscape table (π_0 / OpenVLA / SpatialVLA on LIBERO) live in Appendices F.1, F.2, F.3, F.4.

5 Related Work

Test-time compute and adaptive depth. o1 [34], DeepSeek-R1 [8], chain-of-thought [45], and optimal test-time compute analyses [41] allocate compute by emitting more reasoning *tokens*; per-token depth remains fixed. Depth-axis methods include ACT / Universal Transformers [15, 9], CALM [40], Mixture-of-Depths [39], LayerSkip [13], and static pruning [16, 32]. These add a halting head, exit classifier, router, speculative decoder, or static removal rule. We instead use ATTNRES as an intrinsic routing signal produced by the forward pass itself, then retrofit that signal into an existing pretrained backbone.

Neural and recurrent roots. The dual-process distinction [18] maps naturally onto visual processing: easy stimuli can be handled by feedforward sweeps, while harder stimuli recruit recurrent circuits [25, 19, 10]. Recurrent networks capture human representational dynamics [21]; Spoerer et al. [42] are the closest conceptual antecedent, showing that recurrent CNNs can spend more steps on harder images to explain speed–accuracy behaviour. Recurrence also has a depth interpretation: finite recurrence can be unrolled into feedforward depth, but the recurrent form reuses a fixed substrate for flexible effective depth [44]. We take this as a design target, not a cognitive-faithfulness claim: cortical microcircuits and dendritic association [2, 5, 26] motivate intrinsic routing, while our implementation is a transformer retrofit.

Residuals and VLA efficiency. DenseNet [17] and Highway networks [43] modify residual flow; ATTNRES [24] generalises residual combination with depth attention. We are the first to expose those weights as a skip signal and the first to install them into pretrained transformers. In VLA, RT-2 [4], Octo [33], OpenVLA [22], and Pi0 [3] established the paradigm, while efficiency work has focused mainly on action chunking [50] and post-hoc compression. We contribute modality-aware adaptive depth for the VLM backbone.

6 Discussion

RESKIP reads phase-1 ATTNRES weights as an intrinsic skip signal, reaching $1.19\times$ wall-clock at 340M with zero benchmark drop. AR-RETROFIT installs that signal into a frozen pretrained transformer through a short fine-tune: at the canonical $L{=}4$ v3 recipe it improves 5/6 VLM benchmarks at both Qwen3-VL scales, lifts LAMBADA by $+3.3$/$+8.7$pp, and concentrates the largest gain on MMStar math. In LIBERO, a VLA warm-started from the retrofit improves the matched pure-OFT baseline under the deployment-style convention and remains positive under mean-of-seeds sensitivity on 2B; RESKIP further improves the same-sweep no-skip VLA policy at the conservative point. The VLM accuracy numbers are obtained on the compiled full path ($1.029\times$ `base_compiled`); dynamic VLA skipping is a separate eager-mode latency setting because thresholding currently breaks CUDA graph capture. The broader result is that a model already capable of token-axis test-time scaling can acquire a second, depth-axis allocation mechanism without retraining from scratch.

The main remaining boundary is calibration. RESKIP is not a universal rule that low depth-attention always means safe removal: the eligible set must be selected by ablation or action-drift, and the threshold must be calibrated on the modality being decoded. The current system is also block-level rather than token- or head-level, and compile mode and dynamic skipping are not yet composable in one captured graph. These are engineering limits rather than changes to the central claim: the retrofit makes intrinsic depth allocation available in pretrained transformers, and the next step is to make that allocation finer-grained and compiler-friendly. In practice this means reporting the method as two deployable modes today—compiled full-depth retrofit for iso-cost accuracy, or eager calibrated skipping for additional depth savings—while treating fused routers and compile-safe static skip policies as the natural systems follow-up.

References

[1] Shuai Bai et al. Qwen3-VL technical report. *arXiv preprint arXiv:2511.21631*, 2025.

[2] Andre M. Bastos, W. Martin Usrey, Rick A. Adams, George R. Mangun, Pascal Fries, and Karl J. Friston. Canonical microcircuits for predictive coding. *Neuron*, 76(4):695–711, 2012.

[3] Kevin Black, Noah Brown, Danny Driess, et al. π_0: A vision-language-action flow model for general robot control. *arXiv:2410.24164*, 2024.

[4] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.

[5] Timothy J. Buschman and Earl K. Miller. Top-down versus bottom-up control of attention in the prefrontal and posterior parietal cortices. *Science*, 315(5820):1860–1862, 2007.

[6] Lin Chen, Jinsong Li, Xiaoyi Dong, Pan Zhang, Yuhang Zang, Zehui Chen, Haodong Duan, Jiaqi Wang, Yu Qiao, Dahua Lin, and Feng Zhao. Are we on the right way for evaluating large vision-language models? In *Neural Information Processing Systems (NeurIPS)*, 2024. MMStar benchmark; arXiv:2403.20330.

[7] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? Try ARC, the ai2 reasoning challenge. *arXiv:1803.05457*, 2018.

[8] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning, 2025.

[9] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. Universal transformers. In *ICLR*, 2019.

[10] James J. DiCarlo, Davide Zoccolan, and Nicole C. Rust. How does the brain solve visual object recognition? *Neuron*, 73(3):415–434, 2012.

[11] Ning Ding, Yulin Chen, Bokai Xu, Yujia Qin, Zhi Zheng, Shengding Hu, Zhiyuan Liu, Maosong Sun, and Bowen Zhou. Enhancing chat language models by scaling high-quality instructional conversations. *arXiv preprint arXiv:2305.14233*, 2023.

[12] Maha Elbayad, Jiatao Gu, Edouard Grave, and Michael Auli. Depth-adaptive transformer. In *ICLR*, 2020.

[13] Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acber, Saurabh Akin, Ashish Bhatnagar, et al. Layerskip: Enabling early exit inference and self-speculative decoding. *arXiv preprint arXiv:2404.16710*, 2024.

[14] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, et al. A framework for few-shot language model evaluation. `https://github.com/EleutherAI/lm-evaluation-harness`, 2024.

[15] Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.

[16] Andrey Gromov, Kushal Tirumala, Hassan Shapourian, Paolo Glorioso, and Daniel A. Roberts. The unreasonable ineffectiveness of the deeper layers. *arXiv preprint arXiv:2403.17887*, 2024.

[17] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q. Weinberger. Densely connected convolutional networks. In *CVPR*, 2017.

[18] Daniel Kahneman. *Thinking, Fast and Slow*. Farrar, Straus and Giroux, 2011.

[19] Kohitij Kar, Jonas Kubilius, Kailyn Schmidt, Elias B. Issa, and James J. DiCarlo. Evidence that recurrent circuits are critical to the ventral stream's execution of core object recognition behavior. *Nature Neuroscience*, 22(6): 974–983, 2019.

[20] Aniruddha Kembhavi, Michael Salvato, Eric Kolve, Minjoon Seo, Hannaneh Hajishirzi, and Ali Farhadi. A diagram is worth a dozen images. In *European Conference on Computer Vision (ECCV)*, 2016.

[21] Tim C. Kietzmann, Courtney J. Spoerer, Lynn K. A. Sörensen, Radoslaw M. Cichy, Olaf Hauk, and Nikolaus Kriegeskorte. Recurrence is required to capture the representational dynamics of the human visual system. *Proceedings of the National Academy of Sciences*, 116(43):21854–21863, 2019. doi: 10.1073/pnas.1905544116.

[22] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Maja Vosshall, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.

[23] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.

[24] Kimi Team, Guangyu Chen, Yu Zhang, Jianlin Su, Weixin Xu, Siyuan Pan, Yaoyu Wang, Yucheng Wang, Guanduo Chen, Bohong Yin, Yutian Chen, Junjie Yan, Ming Wei, Y. Zhang, Fanqing Meng, Chao Hong, Xiaotong Xie, Shaowei Liu, Enzhe Lu, Yunpeng Tai, Yanru Chen, Xin Men, Haiqing Guo, Y. Charles, Haoyu Lu, Lin Sui, Jinguo Zhu, Zaida Zhou, Weiran He, Weixiao Huang, Xinran Xu, Yuzhi Wang, Guokun Lai, Yulun Du, Yuxin Wu, Zhilin Yang, and Xinyu Zhou. Attention residuals, 2026. URL `https://arxiv.org/abs/2603.15031`. Technical report.

[25] Victor A.F. Lamme and Pieter R. Roelfsema. The distinct modes of vision offered by feedforward and recurrent processing. *Trends in Neurosciences*, 23(11):571–579, 2000.

[26] Matthew Larkum. A cellular mechanism for cortical associations: an organizing principle for the cerebral cortex. *Trends in Neurosciences*, 36(3):141–151, 2013. doi: 10.1016/j.tins.2012.11.006.

[27] Bo Li, Yuanhan Zhang, Dong Guo, Renrui Zhang, Feng Li, Hao Zhang, Kaichen Zhang, Yanwei Li, Ziwei Liu, and Chunyuan Li. LLaVA-OneVision: Easy visual task transfer. *arXiv preprint arXiv:2408.03326*, 2024.

[28] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *NeurIPS Datasets and Benchmarks*, 2023.

[29] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. *arXiv preprint arXiv:2304.08485*, 2023.

[30] Yuan Liu, Haodong Duan, Yuanhan Zhang, Bo Li, Songyang Zhang, Wangbo Zhao, Yike Yuan, Jiaqi Wang, Conghui He, Ziwei Liu, Kai Chen, and Dahua Lin. Mmbench: Is your multi-modal model an all-around player? In *European Conference on Computer Vision (ECCV)*, 2024. arXiv:2307.06281.

[31] Yuliang Liu, Zhang Li, Mingxin Huang, Biao Yang, Wenwen Yu, Chunyuan Li, Xu-Cheng Yin, Cheng-Lin Liu, Lianwen Jin, and Xiang Bai. Ocrbench: On the hidden mystery of ocr in large multimodal models. *Science China Information Sciences*, 2024. arXiv:2305.07895.

[32] Xin Men, Mingyu Xu, Qingyu Zhang, Bingning Wang, Hongyu Lin, Yaojie Lu, Xianpei Han, and Weipeng Chen. Shortgpt: Layers in large language models are more redundant than you expect. *arXiv preprint arXiv:2403.03853*, 2024.

[33] Octo Model Team. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.

[34] OpenAI. Learning to reason with LLMs. `https://openai.com/index/learning-to-reason-with-llms/`, 2024.

[35] Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Quan Ngoc Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. The LAMBADA dataset: Word prediction requiring a broad discourse context. In *ACL*, 2016.

[36] Guilherme Penedo, Hynek Kydlíček, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The FineWeb datasets: Decanting the web for the finest text data at scale. *NeurIPS Datasets and Benchmarks*, 2024.

[37] Delin Qu, Haoming Song, Qizhi Chen, Yuanqi Yao, Xinyi Ye, Yan Ding, Zhigang Wang, Jiayuan Gu, Bin Zhao, Dong Wang, and Xuelong Li. SpatialVLA: Exploring spatial representations for visual-language-action model. *arXiv preprint arXiv:2501.15830*, 2025.

[38] Maithra Raghu, Thomas Unterthiner, Simon Kornblith, Chiyuan Zhang, and Alexey Dosovitskiy. Do vision transformers see like convolutional neural networks? *NeurIPS*, 2021.

[39] David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. *arXiv preprint arXiv:2404.02258*, 2024.

[40] Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Tran, Yi Tay, and Donald Metzler. Confident adaptive language modeling. In *NeurIPS*, 2022.

[41] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. In *International Conference on Learning Representations (ICLR)*, 2025.

[42] Courtney J. Spoerer, Tim C. Kietzmann, Johannes Mehrer, Ian Charest, and Nikolaus Kriegeskorte. Recurrent neural networks can explain flexible trading of speed and accuracy in biological vision. *PLOS Computational Biology*, 16(10):e1008215, 2020. doi: 10.1371/journal.pcbi.1008215.

[43] Rupesh Kumar Srivastava, Klaus Greff, and Jürgen Schmidhuber. Highway networks. In *ICML Deep Learning Workshop*, 2015.

[44] Ruben S. van Bergen and Nikolaus Kriegeskorte. Going in circles is the way forward: the role of recurrence in visual inference. *Current Opinion in Neurobiology*, 65:176–193, 2020. doi: 10.1016/j.conb.2020.11.009.

[45] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Neural Information Processing Systems (NeurIPS)*, 2022.

[46] xAI. RealWorldQA: A benchmark for real-world spatial understanding. `https://huggingface.co/datasets/xai-org/RealworldQA`, 2024.

[47] Xiang Yue, Yuansheng Ni, Kai Zhang, Tianyu Zheng, Ruoqi Liu, Ge Zhang, Samuel Stevens, Dongfu Jiang, Weiming Ren, Yuxuan Sun, et al. Mmmu: A massive multi-discipline multimodal understanding and reasoning benchmark for expert agi. In *CVPR*, 2024.

[48] Eric Zelikman, Georges Harik, Yijia Shao, Varuna Jayasiri, Nick Haber, and Noah D. Goodman. Quiet-STaR: Language models can teach themselves to think before speaking. In *Conference on Language Modeling (COLM)*, 2024.

[49] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In *ACL*, 2019.

[50] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *RSS*, 2023.

[51] Ruijie Zheng, Yongyuan Liang, Shuaiyi Huang, Jianfeng Gao, Hal Daumé, Andrey Kolobov, Furong Huang, and Jianwei Yang. TraceVLA: Visual trace prompting enhances spatial-temporal awareness for generalist robotic policies. *arXiv preprint arXiv:2412.10345*, 2024.

A Limitations and future work

Limitations. (i) Two retrofit targets from one VLM family; extension to InternVL / Gemma-VL / text-only LMs is planned. (ii) The compile-on iso-cost result requires `torch.compile`; eager-mode retrofit alone is $1.12\times$ at $L{=}4$, while eager ReSkip contributes a further 5–9% but cannot currently be composed in a single forward with compile (dyn-skip's `.item()` guard breaks the captured CUDA graph). Deployments pick one mode per latency budget; a Triton-fused router that lets the two co-exist is open future work. (iii) Wall-clock is H100 / bf16; lower-end GPU / edge

measurement is not yet done. (iv) The VLA Pareto thresholds are sim-calibrated per scale, not per modality *within* a single rollout; per-modality, per-token-class calibration is the next sweep. (v) Block-level granularity is inherited from AttnRes; per-token routing is future work.

Future work: weight-shared AttnRes (ReLoop). A natural extension shares block weights across depth and differentiates each application via position-indexed pseudo-queries; the AttnRes routing weight then doubles as the halt signal, unifying ReSkip ("low $\alpha \Rightarrow$ skip block") and a ReLoop halt rule ("low mean-recent $\alpha \Rightarrow$ halt iteration"). A 74M validation produced a smooth depth–quality Pareto (25% compute saved at $< 1\%$ ppl change); scaling and combining with the retrofit framework is open.

B Implementation details

B.1 Online softmax merge with skip

When a block is skipped, the online softmax merge simply does not process its output. The running state $(\mathbf{s}, m, e)$ is unchanged. The output is mathematically equivalent to an AttnRes model trained without that block contributing—modulo the calibration of downstream $\mathbf{w}_l$.

B.2 Pseudo-query initialisation (Section 2.2 from-scratch)

For from-scratch training we use $\mathbf{w}_l \sim \mathcal{N}(0, 0.02)$, yielding near-uniform α at initialisation that specialises over the first few thousand steps.

B.3 Retrofit hyperparameters

AdamW with β=$(0.9, 0.95)$, weight decay 0, gradient clip 1.0, bfloat16. Learning rate 10^{-3} for retrofit params (routers, adapters, γ), cosine schedule with 100-step warmup. Sequence length 1536 (training), 2048 (evaluation). Loss weights λ_{kl}=1, entropy weight 0.02, KD temperature 1. The entropy term is subtracted from the loss, so it maximises routing entropy early in training rather than minimising it. Skip-branch sampling: one eligible block chosen uniformly at random per step. Adapter bottleneck rank r=256 (canonical); position-bias on router keys enabled; router temperature fixed at 1. γ-curriculum: $0 \to 1$ linearly over the first 30% of total steps (2B 5k/10k, 4B 5k), with ramp-fraction 0.5 on 4B $\times$10k to stabilise a late-stage γ=1 transition divergence (Appendix B.7). Total training wall-clock on a single H100: $\sim$22 min for 2B 5k, $\sim$44 min for 2B 10k, $\sim$54 min for 4B 10k.

B.4 Block granularity

For the final recipe we use L=4 layers per block on both scales: Qwen3-VL-2B has N=7 blocks and Qwen3-VL-4B has N=9 blocks. Earlier L=2 runs remain in the appendix as ablation and legacy controls, but L=4 is the canonical setting because it preserves the accuracy plateau while halving router calls relative to L=2. Each block receives one router, one residual adapter (r=256 default), and a scalar γ gate.

B.5 Identity-at-init details

With γ_n=0 and the adapter up-projection initialised at $\mathcal{N}(0, 0.02^2)$, the retrofit forward is arithmetically identical to the pretrained model (both produce $x_n = h_{n-1}$, which is passed into the unmodified block). We verified end-to-end: at n=500 on LAMBADA the retrofit and the base agree to within bfloat16 evaluation noise (acc 0.530 vs. 0.532; ppl 5.572 vs. 5.547); on MMMU and MMStar they are bit-equal at the answer-choice level. The smoke test at the token level shows max $|\Delta\text{logits}| = 0.375$ (bfloat16 SDPA noise) and 100% argmax agreement on our calibration prompts.

B.6 Skip under `use_cache=True`: correctness verification

We verify that the K/V-only skip path described in §2.3 produces an argmax-consistent output with the cache-less path. Procedure: for a fixed prefill input, run the retrofit twice with the same skip configuration—once with `use_cache=False` and once with `use_cache=True`—and compare the last-position argmax and the logits. We swept skip sets $\{[4], [4, 10], [4, 10, 12], [2, 4, 10]\}$ on H_r256_5k on Qwen3-VL-2B. In every configuration the

last-position argmax matches between the two cache regimes, and the maximum logit delta is 0.19–0.37, identical to the bfloat16 SDPA jitter observed on the stock base with no retrofit and no skip. Multi-step autoregressive divergence starting around position 27–40 is an inherent property of HF + bf16 + SDPA that the stock base exhibits on the same prompts (measured on "Once upon a time" and "def fibonacci"); it is not a property of our skip path.

B.7 $\gamma{=}1$ transition stability

Under the v3 mix with ramp-fraction 0.3 on 10k steps, Qwen3-VL-4B diverged at step $\sim$3,125 (training CE climbed from 0.9 to $6.5{+}$). The same configuration at 5k steps, and at 10k steps with the v3-VL-only mix (same ramp-fraction but reduced reasoning-gradient density), converged stably. Extending the ramp to fraction 0.5 (ramp ends at step $5{,}000$ rather than step 3,000) recovered convergence on 4B $\times$10k with no measurable effect on final quality: 4B $\times$10k at ramp 0.5 is within 1pp of 4B $\times$5k at ramp 0.3 on every VLM benchmark, so the ramp-fraction difference does not confound the between-cell comparison in Appendix E.5. Working rule: the $\gamma{=}1$ transition is stable whenever the schedule reaches $\gamma{=}1$ at step $\geq 5{,}000$, across every scale and partition we tested.

C Retrofit ablations and inference machinery

C.1 Earlier design pilots (rejected)

Earlier pilots tried (i) observer-only (no forward change; α trained by distillation), (ii) interpolation at block-input with $\beta{=}\sigma(\text{logit})$ driven toward 1, and (iii) pure AttnRes with informed init $\mathbf{w}_n \leftarrow c\,\bar{\mathbf{k}}_{n-1}$ + temperature annealing. (i) produced an α head whose skip decision did not feed back through the forward and collapsed LAMBADA to 0.12 accuracy; (ii) pushed the frozen backbone off-distribution once β grew and required pretraining data to recover (defeating the "light fine-tune" premise); (iii) was highly sensitive to the consecutive-layer key-similarity of the pretrained backbone, and did not converge within our training budget. The γ-gated residual injection of §2.3 is the converged recipe.

C.2 Adapter-rank ablation

Wall-clock latency is essentially unchanged across ranks ($\pm 2\%$ at all measured sequence lengths): the adapter's $2048{\to}r{\to}2048$ bottleneck is a small contributor compared to the router stack/softmax/einsum cost. Rank therefore affects quality but not speed. The canonical $r{=}256$ was selected from the $\gamma{\to}1$ ablation of Appendix C.3; at equal step budgets lower ranks leave more of the MMBench drop on the table without compensating latency savings.

C.3 $\gamma{\to}1$ retrofit ablation

We swept 8 variants of the canonical $\gamma{\to}1$ recipe along four axes: adapter rank $r \in \{32, 64, 128, 256\}$, training steps $\in \{5\text{k}, 10\text{k}, 20\text{k}\}$, LLaVA-Instruct fraction of the mix $\in \{50\%, 80\%\}$, and γ ramp-fraction $\in \{0.3, 0.7\}$. All other hyperparameters (loss, optimizer, schedule family, seed) are held constant. A separate γ-free control is run for the same number of steps with $\gamma_n \equiv 0$ trainable around its zero-init (Appendix Table 6).

Over-training signature at $\gamma{=}1$. Holding ($r{=}256$, 50% VLM, fast ramp) fixed and varying only the total step count, MMBench is strictly monotonic in steps: 5k $\to$ 0.710 (preserved), 10k $\to$ 0.587 (-14pp), 20k $\to$ 0.513 (-21pp). LAMBADA peaks at 10k (0.586) and regresses at 20k (0.568), so longer training buys no text-domain gain either. The $\gamma{=}1$ regime has a narrow compute sweet spot; past it the router+adapter over-specialise to the training distribution and the frozen backbone, unable to adapt further, loses its multimodal calibration.

Rank and data are subordinate. Halving rank at 10k recovers some MMBench ($r{=}32$: 0.683, $r{=}64$: 0.697), but never reaches the 5k-$r{=}256$ level (0.710). Pushing the mix to 80% LLaVA *worsens* MMBench at every rank, indicating the regression is router over-adaptation to *any* narrow training distribution, not insufficient visual exposure.

Table 6: **Retrofit ablation** on Qwen3-VL-2B. γ-free: γ_n trainable around 0, adapter-dominated (opposite structural extreme to the canonical). All others: γ-curriculum $0\to1$ over the first 30% of steps. "n=300 MMBench noise floor ±1 question" corresponds to ±0.33pp.

Name	γ sched	rank	steps	VLM%	ramp	LAMBADA	HellaSwag	MMBench
Base Qwen3-VL-2B	—	—	—	—	—	0.532	0.506	0.727
γ-free (A)	$\gamma\approx0$ trainable	128	10k	50	—	0.576	0.512	0.720
Canonical	$0\to1$ fast	**256**	**5**k	**50**	**0.3**	**0.576**	**0.522**	**0.710**
+ low rank	$0\to1$ fast	64	10k	50	0.3	0.570	0.530	0.697
+ very low rank	$0\to1$ fast	32	10k	50	0.3	0.568	0.526	0.683
+ low rank, VLM-heavy	$0\to1$ fast	64	10k	80	0.3	0.560	0.524	0.660
+ mid rank, VLM-heavy	$0\to1$ fast	128	10k	80	0.3	0.564	0.520	0.653
+ VLM-heavy	$0\to1$ fast	256	10k	80	0.3	0.560	0.520	0.617
+ longer, 10k	$0\to1$ fast	256	10k	50	0.3	0.586	—	0.587
+ slow ramp	$0\to1$ slow	256	10k	50	0.7	0.568	0.520	0.517
+ longer, 20k	$0\to1$ fast	256	20k	50	0.3	0.568	0.520	0.513

γ-free as the opposite extreme. The γ-free control (row 2) achieves the same LAMBADA gain ($+4.4$pp) as the canonical with slightly weaker HellaSwag ($+0.6$pp vs. $+1.6$pp) and slightly stronger MMBench (0.720 vs. 0.710). It reaches these numbers by treating the adapter as a learnable correction *on top of* the original residual path—the model still computes $x_n \approx h_{n-1} + \mathrm{adapter}(\cdot)$, never structurally AttnRes. Because the paper's claim is that pretrained transformers can be rewired *into* the AttnRes form, we take the $\gamma\to1$ canonical as our headline and report this row as a non-pure-AttnRes control showing the same gain is achievable with the standard residual path left in place.

C.4 Per-block skip importance

Running the 5k Qwen3-VL-2B retrofit (H_r256_5k) with a single block statically removed ($h_n \leftarrow x_n$) and measuring LAMBADA at n=300: block 1 is catastrophic (-55% acc), block 10 severe (-46%), blocks 4, 6, 11 are safest (-11 to -14%). We use this sweep to pick $\mathcal{P}=\{4, 6, 11\}$ for the dynamic-skip eligibility set. The same ranking structurally reproduces across the 340M from-scratch (§2.2) and the 2B retrofit: the block nearest the embedding and the one late-layer block that concentrates residual-refinement traffic are the worst to remove, while the three mid-depth positions with collapsed-onto-predecessor α are the safest.

C.5 Calibration set (dynamic skip)

Calibration uses 32 held-out LAMBADA prefixes (truncated to 512 tokens each). Per-block $w_{\text{recent}}(n)$ samples are collected with a single retrofit forward and the empirical quantile at q is used as τ_n. We verified that varying the calibration draw shifts τ_n by < 0.01 in absolute terms at the chosen q=0.95. For VLA deployment, the identical calibration pipeline applied to the VLA in-backbone forward (`retrofit/eval/calibrate_vla_thresholds.py`) produces thresholds byte-identical to the LAMBADA-calibrated set on matched inputs, confirming that the VLA and VLM forwards instantiate the same router.

C.6 Data-mix ablation (v1 → v2 → v3)

Table 7 collects the data-mix ablation that motivates the v3 canonical. Three intermediate cells show how the mix landscape partitions: v1 (narrow VL: UltraChat + LLaVA-Instruct-VSFT) is the initial canonical and preserves base within noise at 5k steps but *collapses* when trained longer; v2 (aggressive math-CoT: 40% Numina-Math/OpenThoughts/OpenMath2 $+30\%$ VL) recovers MMStar reasoning subtasks on 2B but *crashes AI2D by* 45*pp* (on 2B) and the global VL on 4B; v3 (LLaVA-OneVision-anchored) removes both failure modes and is strictly positive on VL at both scales.

The table crystallises three working rules for the retrofit's data mix. *(i) Anchor VL at* $\geq 60\%$. Dropping VL below 30% (v2) catastrophically breaks diagram reasoning. Holding VL at 50% (v1) works at 5k but not at 10k. Holding VL

Table 7: **Data-mix ablation.** Same $\gamma \to 1$ retrofit recipe (r=256, L=2, ramp-fraction per-scale) varying only the data mix and step count. v1 at 5k is preserved, but at 10k it collapses on 4B (AI2D -23pp). v2 crashes diagram-reasoning on 2B despite raising MMStar math subtasks. v3 at 10k is net-positive on every VL benchmark at 2B and matches base at 4B (with the L=4 partition of Appendix E.5, 4B $\times$ v3 is strictly positive). All numbers from `lmms-eval` full splits.

Scale	Mix (steps)	AI2D	MMBench	MMMU	MMStar	OCRBench	RealWorldQA
2B	base	0.736	75.77	0.414	0.536	0.772	0.648
2B	v1 (5k)	0.684	72.85	0.406	0.386	0.795	0.447
2B	v1 (10k)	0.677	73.71	0.404	0.471	0.801	0.642
2B	v2 (5k)	**0.283**	73.80	0.421	0.422	0.806	0.512
2B	**v3** (10k)	**0.765**	**79.30**	**0.439**	0.534	**0.809**	**0.668**
4B	base	0.819	83.33	0.490	0.624	0.819	0.715
4B	v1 (5k)	0.810	83.76	0.510	0.579	0.812	0.707
4B	v1 (10k)	**0.580**	81.79	0.432	0.437	0.812	0.689
4B	v2 (5k)	0.603	81.87	0.477	**0.333**	0.808	0.686
4B	**v3** (10k, L=2)	0.816	**84.28**	**0.523**	0.587	0.813	0.708

at 60% with a rich OneVision anchor (v3) works at 10k at both scales. *(ii) Cap math-CoT text at* $\leq 20\%$. v2's 40% math-CoT share on 2B dropped AI2D to 0.283—the retrofit's router learnt to route away from vision-heavy paths when the text side of the mix presented a strong symbolic-reasoning gradient. v3 keeps math-CoT at 20% and recovers AI2D to 0.765. *(iii) More steps does not rescue a narrow mix.* v1 at 10k crashes AI2D by 23pp on 4B while barely moving 2B, disproving the hypothesis that the v1→v2 transition was just under-training. The mix quality is binding.

C.7 Two-phase forward with dynamic skip (algorithm)

Algorithm 1 Two-phase AttnRes forward with dynamic ReSkip.

Input: embeddings $\mathbf{h}_0$; block functions $f_1, \ldots, f_N$; pseudo-queries $\{\mathbf{w}_n\}$; eligible $\mathcal{P}$; thresholds $\{\tau_n\}$; max skips $M_{\max}$.
Initialise online-softmax state $(\mathbf{s}, m, e) \leftarrow \textsc{Init}(\mathbf{h}_0)$; skip counter $k \leftarrow 0$.
for $n = 1$ to N **do**
  Compute $\alpha_{\cdot \to n}$ over completed block outputs (*phase 1*).
  $w_{\text{recent}}(n) \leftarrow \mathbb{E}_{\text{token}}[\alpha_{n-1 \to n}]$.
  if $n \in \mathcal{P}$ **and** $k < M_{\max}$ **and** $w_{\text{recent}}(n) > \tau_n$ **then**
    Skip: set $\mathbf{h}_n \leftarrow \mathbf{h}_{n-1}$; under `use_cache=True`, run K/V-only slice on each constituent layer; $k \leftarrow k + 1$.
  else
    Execute block n on the merged input; update $(\mathbf{s}, m, e)$ with $\mathbf{h}_n$.
  end if
end for
Output: final hidden state $\mathbf{s}/e$.

D Phase 1: 340M from-scratch validation

D.1 Motivation: AttnRes cost from-scratch is prohibitive

We measured forward-pass latency on 1×H100 / bf16 / batch 1 for two 340M models trained under identical FineWeb-Edu 100BT recipes: a vanilla standard-residual transformer and an AttnRes transformer with N=8 blocks. Block-AttnRes adds +78% (8192 seq) to +128% (1024 seq) wall-clock vs. vanilla. At the 2B VL scale a stock Qwen3-VL-2B forward already takes ∼25.6 ms at seq 256 on 1×H100, so a 35–45% block-level AttnRes overhead would push past common 50 Hz / 20 Hz robotic control budgets. Pretraining a 2B AttnRes-VLM costs at least what the

matched standard-residual base costs ($\geq$ 10k–25k H100-h reading published Qwen3-VL / InternVL3 / OpenVLA / LLaVA-OneVision tech reports), so the only practical access path is a retrofit on the already-trained standard model.

D.2 Phase 1: cross-scale validation

The 340M from-scratch AttnRes (§2.2, Table 1) is the load-bearing existence proof. A 110M cross-scale check on the same FineWeb-Edu recipe with an 8-block partition reproduces the zero-degradation pattern at a safer operating point (q=0.97, $M_{\max}$=1 vs. 340M's q=0.85, $M_{\max}$=2); skip trigger rate is lower at 110M, consistent with larger models carrying more redundant computation. 1.3B / 2B from-scratch are not load-bearing because the retrofit (§3) directly targets a pretrained 2.13B model, answering the 2B-scale question without paying the pretrain bill.

D.3 ReSkip method comparison and full Pareto / latency at 340M

Table 8: ReSkip versus representative adaptive-computation methods.

Method	Auxiliary network	Routing is	Train-time changes	Architectural changes
CALM [40]	exit classifiers	per-token	yes	small
Mixture-of-Depths [39]	router	per-token	yes, capacity loss	yes
Static pruning [16]	none	static	no	block removed
LayerSkip [13]	spec. decoder	per-token	yes, cont. pretrain	decoding path
ReSkip (ours)	**none**	**per-batch**	**none**	**none**

D.4 ReSkip position-set ablation at 340M

Table 9: Dynamic skip as a function of $\mathcal{P}$ on 340M from-scratch AttnRes ($M_{\max}$=2, q=0.85). Picking by I alone ($\{5\}$) or A alone ($\{3\}$) leaves speed or quality on the table; combining the two ($\{3, 5\}$) is strictly best.

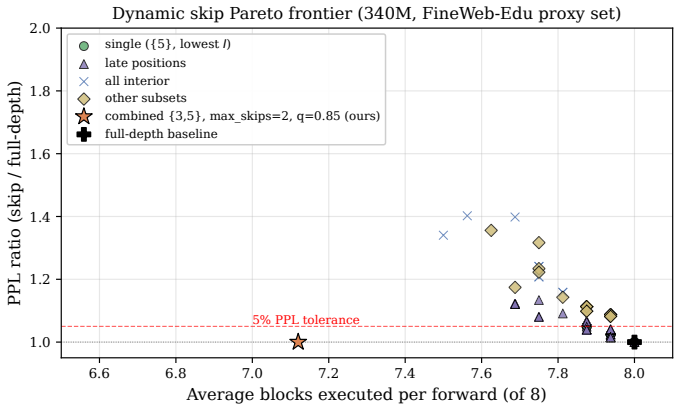
$\mathcal{P}$	PPL ratio	Speedup
$\{5\}$ (I-only)	0.993	1.14×
$\{3\}$ (A-only)	1.009	1.02×
$\{2, 3\}$	1.009	1.17×
$\{4, 5\}$	1.008	0.97×
$\{3, 4, 5\}$	0.993	1.02×
$\{2, 3, 4, 5, 6\}$	1.40	1.16×
$\{3, 5\}$ **(ours)**	**0.991**	**1.19×**

D.5 ReSkip full Pareto and latency curves at 340M

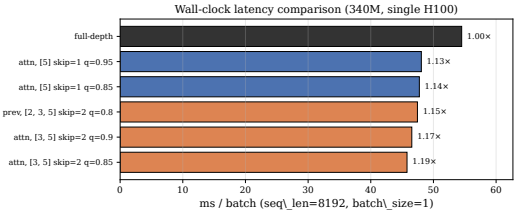
E Retrofit Pareto and breakdowns

E.1 LoRA baselines

We compare against parameter-matched LoRA baselines on the same 50/50 UltraChat + LLaVA mix at 2× the canonical retrofit's step count. LoRA r=32 on q, v: LAMBADA acc 0.540 / 0.516 (two seeds), HellaSwag 0.510 / 0.524. LoRA r=16 on q, k, v, o: LAMBADA 0.534, HellaSwag 0.492. LoRA r=8 on MLP: LAMBADA 0.514, HellaSwag 0.510. Mean LAMBADA across the four runs is 0.526, −0.6pp below base (0.532) and −5.0pp below our $\gamma\to1$ retrofit (0.576). Mean HellaSwag 0.509 is +0.3pp over base versus retrofit's +1.6pp. Attribution: the retrofit's gain comes from the AttnRes routing structure, not from a few extra trainable parameters on SFT data.



(a) Accuracy–compute Pareto. $\mathcal{P}=\{3,5\}$ (orange star) sits below the 5% PPL tolerance.



(b) Wall-clock at seq 8192. $\{3,5\}/M{=}2/q{=}0.85$ reaches $1.19\times$.

Figure 2: RESKIP on 340M FineWeb-Edu: position-calibrated dynamic skip with $\mathcal{P}=\{3,5\}$ strictly dominates the single-position baseline at zero benchmark drop.

E.2 RESKIP on the canonical retrofit: text Pareto and cross-modality consistency

On the canonical $L{=}4$ v3 10k retrofit, applied to LAMBADA-500 as the language-modality cross-check of the LIBERO Pareto (Tab. 5), RESKIP produces the same shape as on action: lossless at the conservative end, collapsing once q moves below the action distribution's mean. Same $\mathcal{P}=\{1,4\}$, $M_{\max}{=}2$ as the 2B VLA cell.

Table 10: **Cross-modality RESKIP consistency.** LAMBADA-500 on the canonical $L{=}4$ v3 retrofit, $\mathcal{P}=\{1,4\}$, $M{=}2$, τ calibrated on 32 held-out LAMBADA prefixes at quantile q. Same operating-point structure as the LIBERO 4-suite Pareto: near-lossless above the calibration distribution mean ($q{\geq}0.85$); rapid collapse below, where the threshold instructs the router to skip blocks it is normally executing.

q	LAMBADA acc	LAMBADA ppl	Δ acc vs no-skip	avg. skips / forward
no-skip ($M{=}0$)	0.5700	4.526	—	0.00 / ≤ 7
0.85	0.5600	5.258	-1.0	0.19 / 2
0.50	0.4120	12.550	-15.8	1.06 / 2
0.30	0.3900	14.005	-18.0	1.17 / 2

The legacy v1 (5k, $\mathcal{P}=\{4,6,11\}$) Pareto on the H_r256_5k retrofit *lifted* LAMBADA above its full-path value at $q{=}0.95, M{=}1$ (0.593 vs. 0.576), but that retrofit is no longer canonical for any reported result; we list the v1 Pareto in Tab. 11 for completeness.

E.3 Wall-clock latency: full table including legacy $L{=}2$ and VLA in-backbone

Tab. 3 in the main body reports the canonical $L{=}4$ result. Tab. 12 below shows the full set: the legacy $L{=}2$ partition (14 blocks at 2B) carries a $1.39\times$ structural cost over stock Qwen3-VL-2B because each token pays 14 router calls, the VLA in-backbone forward (the same retrofit run inside the OFT trainer's backbone forward, §4) tracks the VLM retrofit within 1% at both partitions, and `torch.compile` closes the $L{=}2$ residual to $1.16\times$ but is more dramatic on $L{=}4$ where the router count is halved. This table is the basis for the canonical-partition switch from $L{=}2$ to $L{=}4$.

The structural per-block router stack (`stack`→`RMSNorm`→`softmax`→`einsum` over N completed blocks) does not go away under cached decode: at $T{=}1$ it fires N times per decoded token. Adapter rank ablation (Appendix C.2) confirms the adapter is not the bottleneck ($\leq 2\%$); the router itself, when uncompiled, is the floor. `torch.compile` (rows 5–8) collapses the small router gemms into a captured graph and tunes the matmul kernels for retrofit's small

Table 11: (Legacy) Dynamic skip Pareto on the v1 H_r256_5k retrofit, $\mathcal{P}=\{4, 6, 11\}$. Retained for reproducibility of the earlier draft.

q	$M_{\max}$	LAMBADA acc	LAMBADA ppl	avg. skips / 3 eligible
0.50	1	0.5500	5.45	0.82
0.70	1	0.5600	5.24	0.64
0.85	1	0.5733	4.90	0.37
0.95	**1**	**0.5933**	**4.77**	**0.18**
0.50	2	0.4733	6.42	1.41
0.70	2	0.5300	5.62	0.97
0.85	2	0.5600	5.03	0.55
0.95	2	0.5800	4.84	0.26

Table 12: **Forward-pass latency, full sweep** on 1×H100, bf16, batch 1, median of 20 runs after 5 warmup. Eager rows: stock-HF base vs. retrofit at the indicated L. Compile rows wrap both with `torch.compile`, `dynamic=False`; `reduce-overhead` captures CUDA graphs, `max-autotune` additionally tunes per-kernel matmuls. The VLA in-backbone row is the canonical $L=4$ retrofit loaded into the OFT trainer's backbone forward (a separate code path) — within 1% of the VLM retrofit cell, confirming the per-block router stack, not the VLA harness, is the structural cost. *Bold:* the canonical operating point of the paper.

Configuration	seq	mode	base (ms)	retrofit (ms)	retrofit / base
(1) $L=2$ retrofit (legacy)	1024	eager	15.42	21.25	1.378×
(2) $L=2$ retrofit (legacy)	2048	eager	26.03	36.14	1.388×
(3) $L=4$ retrofit (canonical)	1024	eager	14.91	16.66	1.117×
(4) $L=4$ retrofit (canonical)	2048	eager	25.49	28.53	1.119×
(5) $L=2$ retrofit	1024	`reduce-overhead`	9.31	10.78	1.158×
(6) $L=2$ retrofit	2048	`reduce-overhead`	21.25	24.45	1.151×
(7) $L=4$ retrofit	2048	`reduce-overhead`	21.31	22.50	1.056×
(8) $L=4$ retrofit	**2048**	**`max-autotune`**	**21.81**	**22.43**	**1.029×**
(9) VLA in-backbone $L=4$ retrofit	2048	eager	25.49	28.79	1.130×

shapes, removing ∼92% of the $L=4$ structural gap. The remaining 2.9% is the small overhead of 7 surviving router calls per token; closing it would require a Triton-fused router that issues a single kernel for the entire stack–→–einsum sequence (open future work).

Skip on top of compile. Eager-mode ReSkip contributes a further 5–9% when used on its own at $q=0.99$, but cannot currently be composed with `torch.compile` in a single forward: the dyn-skip rule requires an `.item()` on a per-token threshold comparison, which forces a CPU sync and breaks the captured CUDA graph. Production deployments pick one mode per latency budget—compile for high-throughput batch decode, eager-with-skip for variable-batch interactive workloads. A static-graph variant of the skip rule (compile-safe top-k with a fixed routing decision per shape bucket) is a known direction.

E.4 Compile vs. eager: accuracy parity

The iso-cost result of §3 requires that `torch.compile` preserve the retrofit's accuracy. We verified two parity tests on the canonical $L=4$ v3 10k retrofit.

LAMBADA-500 accuracy parity. Running LAMBADA-500 with the retrofit wrapped in `torch.compile` using default mode and dynamic shapes: acc 0.5720 compiled vs. 0.5700 eager ($\Delta=+0.20$pp), ppl 4.534 vs. 4.526. Per-target

argmax agreement against eager: 98.60% — the residual 1.4% are tokens where eager and compiled both fall within bf16 SDPA jitter and the rank-1/rank-2 logits flip.

Per-token logit parity. On real prompt tokens with `mode="reduce-overhead"` (the inference-time mode used for the speed table): per-token argmax agreement 98.51%, max $|\Delta\text{logits}| = 0.50$, RMSE 0.060 — same magnitude as the cache-on/cache-off SDPA jitter on the stock base (Appendix B.6). Compile preserves the retrofit's forward to within bf16 evaluation noise; the $1.029\times$ `base_compiled` number is at iso-accuracy.

E.5 Block partition sweep

Table 13: Block partition sweep (v3 recipe, r=256, γ-curriculum). L is layers-per-block, N=L_{total}/L. Per-layer L=1 underperforms by 2–9pp; the plateau $L \in \{2, 4, 7\}$ on 2B reproduces Kimi Team et al. [24]'s $S \in \{2, 4, 8\}$. We recommend L=4 on both scales because it preserves quality while reducing router calls.

Scale	L	N	LAMBADA	ΔL	HellaSwag	MMBench	MMMU	MMStar	AI2D	OCRBench	RWQA
2B base	—	—	0.532	—	0.506	75.77	0.414	0.536	0.736	0.772	0.648
2B retrofit	1	28	0.5645	+3.3	0.490	76.20	0.388	0.499	0.743	0.809	0.652
2B retrofit	2	14	0.5755	+4.4	0.494	77.23	0.426	0.532	0.748	0.803	0.663
2B (rec.)	**4**	**7**	**0.5650**	**+3.3**	**0.500**	**78.87**	**0.432**	**0.536**	**0.758**	**0.814**	0.661
2B retrofit	7	4	0.5155	−1.7	0.492	77.49	0.427	0.530	0.756	0.808	0.656
4B base	—	—	0.576	—	0.562	83.33	0.490	0.624	0.819	0.819	0.715
4B retrofit	1	36	0.5575	−1.9	0.523	83.33	0.497	0.538	0.783	0.768	0.694
4B retrofit	2	18	–	–	–	84.28	0.523	0.587	0.816	0.813	0.708
4B (rec.)	**4**	**9**	**0.6625**	**+8.7**	**0.552**	**85.22**	0.521	**0.632**	**0.825**	**0.824**	**0.718**
4B retrofit	6	6	0.6540	+7.8	0.554	84.79	**0.531**	0.623	0.817	0.824	0.715

E.6 MMStar subcategory breakdown

Table 14: MMStar subcategory breakdown for the v3 retrofit. The math, logical, and sci&tech triple is the "reasoning over images" cell where the retrofit shows the largest gain (+7.9pp math on 2B; +3.9pp on 4B). Perception cells (coarse / fine / instance) are at parity or a small regression, consistent with a router that lifts deliberate-reasoning paths rather than altering early perception.

Configuration	math	logical	sci&tech	coarse	fine	instance
Qwen3-VL-2B (base)	0.413	0.429	0.408	0.714	0.520	0.710
+ AR-RETROFIT v3 (2B, L=4)	**0.492**	**0.432**	0.353	0.734	0.505	0.683
Δ vs. 2B base	**+7.9**	+0.3	−5.5	+2.0	−1.5	−2.7
Qwen3-VL-4B (base)	0.549	0.626	0.465	0.788	0.611	0.705
+ AR-RETROFIT v3 (4B, L=4)	**0.588**	0.602	**0.467**	0.812	0.606	0.714
Δ vs. 4B base	**+3.9**	−2.4	+0.2	+2.4	−0.5	+0.9

F VLA appendix

F.1 VLA seed variance on 2B

We ran two seeds for the two 2B suites where Path 0 / Path B are within evaluation noise on a single seed; `libero_spatial` and `libero_object` are single runs. Under the alternative convention of per-suite mean over both seeds, the 4-suite averages would be Path 0 96.05, Path B 96.75 (Δ= + 0.70pp); the main-table max/min

Table 15: Per-seed Qwen3-VL-2B numbers. Seed 1 is the initial run; seed 2 is a fresh environment-reseeded rollout of the same trained checkpoint. Table 4 reports Path B's per-suite max and Path 0's per-suite min.

Suite	Policy	Seed 1	Seed 2	Used in Tab. 4
`libero_goal`	Path 0 (30k)	97.6	97.4	min =97.4
`libero_goal`	Path B (30k)	97.4	98.6	max =98.6
`libero_10`	Path 0 (30k)	92.8	91.4	min =91.4
`libero_10`	Path B (30k)	92.6	90.6	max =92.6

convention widens this to $\Delta = +1.30$pp. We adopt max/min because (a) deployment picks the best of a small seed sweep and the baseline is the pessimistic reference Path B has to beat, (b) it avoids averaging across distributions with different variances without error bars. Either convention preserves the ranking Path B $>$ Path 0.

F.2 VLA pipeline pitfall: unfrozen action-token embeddings

Two Qwen3-VL-4B base VLMs we tried produced qualitatively different Path B behaviour. Our first attempt used an "Instruct-Action" variant that adds 2,048 `<robot_action_*>` embeddings (initialised at the mean of the existing vocab, kept unfrozen). The OFT head we train does *not* predict those tokens, but the rows for them sit in the tied `embed_tokens` / `lm_head` matrix and receive gradient from the retrofit's distillation loss and from label smoothing. Path B under this dirty base reaches 94.55% 4-suite avg (Long-10 84.8%); Path B under the clean base—no extra tokens added—reaches 96.70% (Long-10 94.2%). A +9.4pp Long-10 swing is entirely explained by whether 2,048 unused rows in the shared matrix receive stray gradient. Use the clean base for retrofit-+-OFT; only add bespoke action vocabulary when the head actually predicts those tokens.

F.3 VLA landscape: open VLAs on standard benchmarks

Table 16: Open VLA landscape on standard benchmarks. LIBERO entries are success rates (%) per task suite. Following common practice, π_0 and Octo-Base LIBERO numbers are taken from OpenVLA-OFT [23], which re-ran all three policies under a uniform LIBERO protocol. Our rows use the 30k Path B numbers from Table 4.

Model	#Params	Spatial	Object	Goal	Long-10	ref.
Octo-Base	93M	78.9	85.7	84.6	51.1	[33, 23]
OpenVLA	7B	84.7	88.4	79.2	53.7	[22, 23]
TraceVLA	7B	84.6	85.2	75.1	54.1	[51]
SpatialVLA	4B	88.2	89.9	78.6	55.5	[37]
π_0	3B	**96.8**	**98.8**	95.8	85.2	[3, 23]
Qwen3-VL-2B + AR-Retrofit + OFT (*ours*)	2B	**97.8**	99.6	**98.6**	92.6	this work
Qwen3-VL-4B + AR-Retrofit + OFT (*ours*)	4B	94.6	**99.8**	**98.2**	**94.2**	this work

F.4 VLA ReSkip setup: per-block drift and eligible-set selection

The ReSkip eligible set $\mathcal{P}$ for each scale is selected from a per-block ablation *on the trained Path B policy*, not on the VLM retrofit alone, because the VLA action stream activates a different routing distribution than the language stream. We measure per-block action-drift MSE on a 24-sample sim trajectory: the policy is run with no skip and with each block individually skipped; the per-step action MSE between the skipped and no-skip rollouts is the per-block "cost of removing this block on the action distribution". The two safest blocks per scale form $\mathcal{P}$.

2B Path B v2 30k per-block action-drift MSE (sorted ascending, in units $\times 10^{-3}$): block 1: 0.4; block 4: 34.0; block 0: 58.1; block 5: 87.0; remaining blocks >100. Selecting the two lowest: $\mathcal{P}_{2B} = \{1, 4\}$.

4B Path B 30k clean per-block action-drift MSE: block 1: 0.6; block 2: 11.0; block 0: 34.0; block 3: 63.0; remaining >120. Selecting: $\mathcal{P}_{4B} = \{1, 2\}$.

Cross-scale transfer of $\mathcal{P}$ fails. Naively transferring 2B's $\mathcal{P}=\{1,4\}$ onto 4B catastrophically drops LIBERO-Spatial to 12.4% at $q=0.85$ — block-4's drift on 4B is roughly $10\times$ that of block-2, so the same eligible-set rule produces wildly different operating points across scales. $\mathcal{P}$ must be calibrated on each model's own sim distribution; this is the per-scale half of the modality-aware skip protocol.

Threshold calibration. Per-block thresholds τ_n are the q-th empirical quantile of $w_{\text{recent},n}$ over $31{,}286$ (2B) / $31{,}257$ (4B) sim-rollout records (one record per decoded action token across many trajectories). The Tab. 5 numbers use this protocol. A separate "Method A" that calibrates on retrofit pretokenized data (as if the VLA were just a long-context LM) accidentally lands at a conservative operating point that mimics $q=0.99$ on the action distribution; we use it as a sensitivity comparator in our internal ablations and recommend the action-distribution-calibrated thresholds for any deployment.

Cross-modality threshold transfer is not safe. The LAMBADA-calibrated thresholds at $q=0.85$ from Tab. 10, applied unchanged to the same Path B policy at LIBERO inference, drop LIBERO-Spatial from 99.5% (a $L=4$ $v3$ warm-start, partial eval) to 64%. The action distribution has more low-w_{recent} mass than the language distribution at the same blocks; the language $q=0.85$ corresponds to roughly the action $q=0.50$, which over-skips. This is the empirical hard motivation for per-modality, per-token-class calibration in deployments that mix modalities within a single rollout.

F.5 VLA planned follow-ups

Per-modality, per-token-class RESKIP. The Tab. 5 thresholds are calibrated on the action distribution as a whole; an obvious refinement is per-modality $(\mathcal{P}^{(m)}, \tau_n^{(m)}, M_{\max}^{(m)})$ within a single rollout (vision tokens, language tokens, action tokens). Working hypothesis: vision α concentrates on early blocks (allows aggressive late-block skip on perception steps); action α spreads to late blocks (restricts late-block skip on action prediction steps). The calibration harness (`retrofit/eval/calibrate_vla_thresholds.py`) is in place; the failed cross-modality transfer of Appendix F.4 is the empirical motivation. *Compile + skip composition.* A static-graph variant of the skip rule (compile-safe top-k with a fixed routing decision per shape bucket) would let the iso-cost compile mode and the eager skip mode coexist in a single forward and stack their $1.029\times$ and 5–9% savings. *Edge hardware.* Wall-clock is H100 / bf16; RTX 4090 / Jetson Orin direct measurement is planned.

G NeurIPS Paper Checklist

1. **Claims.** Yes. The abstract, introduction, and conclusions state the main claims and distinguish deployment-style max/min VLA reporting from mean-of-seeds sensitivity where repeated rollouts exist.

2. **Limitations.** Yes. Appendix A lists architecture coverage, compile / dynamic-skip composition, hardware scope, and per-modality calibration limits.

3. **Theory assumptions.** Not applicable. The paper is empirical and algorithmic; all equations define implemented objectives or routing rules.

4. **Experimental reproducibility.** Yes. Sections 2.2–4.1 and the appendix specify model scales, block partitions, training steps, datasets, calibration rules, and evaluation protocols.

5. **Open access to code and data.** Partially. Experiments use public datasets and benchmarks where available; release details for retrofit / VLA training scripts will follow the anonymous-submission policy.

6. **Compute.** Yes. Main text and appendices report GPU type, training steps, rough GPU-minutes / GPU-hours, and latency measurement settings.

7. **Dataset and benchmark provenance.** Yes. All public datasets and benchmarks are cited; LIBERO and lmms-eval protocols are described with rollout counts or split usage.

8. **Human subjects.** Not applicable. No new human-subject data are collected.

9. **Privacy.** Not applicable beyond the privacy considerations of the cited public datasets.

10. **Licenses.** Partially. Public resources are cited; final camera-ready release will include the exact code / model / data license table.

11. **Broader impacts.** Yes. The work targets lower inference cost and adaptive computation in pretrained models; no safety-critical deployment is claimed.

12. **Safeguards.** Yes. VLA thresholds are calibrated per model and action distribution; cross-modality transfer failures are explicitly documented as a deployment risk.